

Bookings v2

Frontend Integration Guide

Magskr backend · base path `/bookings/v2` · generated 29 August 2026

First venue: **Tekkers** (football party bookings, GBP)

CONTENTS

1. What this API is (and what it replaces)
2. **If you already know Reservation v2** — what changed
3. The four concepts you need
4. The customer booking flow — 5 calls
5. Endpoint reference
6. Taking payment with Stripe
7. Handling rejections (409)
8. Staff / owner dashboard endpoints
9. Gotchas worth reading before you start

1 · What this API is

A booking engine for **appointment-style venues** — event spaces, salons, doctors, courts. A customer picks a **service** (a package with its own length and price), sees genuinely free time slots, books one, and pays.

It is a **completely separate system** from the restaurant side. Restaurants use `/reservations/v2`; they never appear in this API and nothing here affects them. The older `/bookings` API is separate again. **Do not mix the three.**

The old `/bookings` API is deprecated for new work.

It has no availability checking at all — two customers can book the same hour. Build against `/bookings/v2`. The old endpoints stay live only until the current site is migrated.

All times are **naive Berlin wall-clock**, matching the rest of the backend. Send and read ISO strings without a timezone suffix: `2026-09-15T15:00:00`.

2 · If you already know Reservation v2

This engine started as a copy of the restaurant reservation engine, so the shape will feel familiar. Read this section and you can skim the rest.

What is identical

- Same slot model: opening hours + `slot_interval_minutes` generate a grid, holidays and lead time remove slots.
- Same statuses: `pending` → `booked` / `cancelled`.
- Same route shapes: `/config`, `/availability`, `/guest`, `/my`, `/store/today`, `/store/filter`, `GET|PUT|DELETE /{id}`.
- Same 409-with-`alternatives` pattern when a slot is gone.
- Same naive Berlin wall-clock times.

What changed

TOPIC	RESERVATION V2	BOOKINGS V2
What is booked	implicit — always "a table"	a service the customer picks; <code>service_id</code> is required
Duration	one per store <code>default_duration_minutes</code>	per service — 60 / 90 / 120 min for the three packages
Capacity lives on	the store <code>max_covers</code>	a resource — <code>capacity</code> + <code>capacity_mode</code>
Availability is per	<code>?store_id=</code>	<code>?service_id=</code> ← the main call-site change
Time fields	<code>reserved_for</code> <code>reserved_until</code>	<code>start_at</code> , <code>end_at</code> , plus <code>blocked_until</code> (includes the turnaround buffer)
Turnaround	none	<code>buffer_minutes</code> — blocks the next slot, never shown to the customer
Money	none — reservations are free	price, options, Stripe payment , <code>payment_status</code>
Custom fields	none	<code>form_data</code> — free-form JSON, no backend change to add a question
Guest identifies with	email or phone used when booking	a reference code (<code>BK-7KQ2WM</code>) returned on create
Who confirms	staff accept, or <code>auto_accept</code>	the Stripe webhook when payment is due; staff otherwise
Push notifications	main Firebase project	the separate booking Firebase project

Mapping your old calls across

`party_size`, `customer_name`, `customer_phone`, `customer_email`, `note` and `status` all carry over unchanged. Rename `reserved_for` → `start_at`, swap `store_id` → `service_id` on the availability call, and add `service_id` to the create payload. That is most of the port.

Two habits to unlearn

1. In reservations, capacity is always headcount. Here a booking on an `exclusive` resource consumes 1 unit regardless of party size — so `available: 1, total: 1` is normal for Tekkers and does *not* mean "one seat left". Read `bookable`, never `available`.
2. A reservation is confirmed the moment staff accept it. A *paid* booking is not confirmed until Stripe says so — don't reuse the reservation confirmation UI as-is.

3 • The four concepts

Service — what the customer picks

A bookable package. Carries its own duration, party limits, price and images. For Tekkers there are three: Premier League, Champions League, World Cup.

Resource — what gets used up

The thing a booking occupies. Tekkers has one: **the stadium**. Its `capacity_mode` decides how much of it a booking eats:

MODE	A BOOKING CONSUMES	VENUE TYPE
<code>exclusive</code>	1 unit , whatever the headcount	Tekkers — one party hires the whole pitch. Also a salon chair, a doctor, a tennis court.
<code>pooled</code>	<code>party_size</code> units	venues that count <i>people</i> , not whole-venue hire: a trampoline park with 40 spaces, soft play, a yoga class

Tekkers is `exclusive` only — you can ignore `pooled` .

And you never implement either: `/availability` does the maths and returns `bookable: true/false` per slot. This is here so the numbers make sense, not because you have to act on them.

All three Tekkers packages share the one exclusive stadium. A Premier League booking at 15:00 therefore blocks Champions League at 15:00 too — already handled server-side.

Slot — a bookable start time

The backend generates the grid from the venue's opening hours. **You may only book a start time the server published.** Posting an arbitrary time like 15:07 is rejected.

Buffer — invisible reset time

Each service has `buffer_minutes` (15 for Tekkers) — clean-down between parties. It blocks the next slot but is **never shown to the customer** as part of their session. Show `duration_minutes` only.

Worked example

A 15:00 Premier League booking runs 15:00–16:00 for the customer, but occupies the stadium until 16:15. So 16:00 is unavailable and 16:30 is the next bookable slot.

4 • The customer booking flow

- List the packages**
`GET /services?store_id=42` — render name, description, image, price, duration.
- Customer picks a package + date + headcount**
`GET /availability?service_id=1&date=2026-09-15&party_size=8` — render the slot grid.
- Show the price**
(optional but recommended) `POST /quote` — returns an itemised total, including per-child food options.
- Create the booking**
`POST /guest` — returns the booking with its `reference` code.
Store that reference.
- Take payment**
`POST /{id}/payment-intent?reference=BK-XXXXXX` — returns a Stripe `client_secret` ; confirm it with Stripe.js.

The booking is confirmed by Stripe, not by you.

After a successful create the booking is `status: "pending"` , `payment_status: "pending"` . Our Stripe webhook flips it to `booked` / `paid` . Never mark a booking confirmed client-side.

5 · Endpoint reference

no auth logged-in customer store owner / admin JWT

OWNER CUSTOMER PUBLIC

GET /bookings/v2/services?store_id={id}

PUBLIC

The package list. Add `&include_inactive=true` to include hidden ones.

```
[{
  "id": 1,
  "name": "Premier League",
  "description": "Access to Tekkers Stadium · 2 games · ...",
  "image_url": "https://.../premier.jpg",
  "duration_minutes": 60,      // show this
  "buffer_minutes": 15,      // do NOT show this
  "min_party": 1, "max_party": 10,
  "price_mode": "flat",      // none | flat | per_person
  "price": 150.00,
  "options": [],             // see World Cup below
  "is_active": true, "sort_order": 1
}]
```

Rendering the price

`flat` → show `price` as the total. `per_person` → the total is `price × party_size` plus any chosen option, also per person. The World Cup package uses `per_person` with `price: 0`, so the whole charge comes from the hot/cold food option (£38.75 / £37.95 per child). Always confirm with `/quote` rather than computing it yourself.

GET /bookings/v2/availability?service_id=&date=&party_size=

PUBLIC

`date` is `YYYY-MM-DD`. Returns every slot for that day.

```
{
  "service_id": 1, "date": "2026-09-15", "party_size": 8,
  "enabled": true, "duration_minutes": 60,
  "reason": null,      // set ONLY when slots is empty
  "slots": [
    { "time": "10:00", "datetime": "2026-09-15T10:00:00",
      "available": 1, "total": 1, "bookable": true },
    { "time": "15:00", "datetime": "2026-09-15T15:00:00",
      "available": 0, "total": 1, "bookable": false }
  ]
}
```

Render every slot, but only allow selection where `bookable: true` — showing greyed-out taken slots reads better than hiding them. **Send back the exact `datetime` string**, not one you rebuild from `time`.

When `slots` is empty, `reason` tells you why:

REASON	SHOW THE CUSTOMER
closed	"Closed on this date"
closed_holiday	"Closed — holiday"
party_too_large	"Maximum N children — please call us"
party_too_small	"Minimum N children for this package"
outside_booking_window	"Bookings open N days ahead"
bookings_disabled / service_inactive	"Not bookable online"

POST /bookings/v2/quote

PUBLIC

```
// request
{ "service_id": 3, "party_size": 10, "option_ids": [1] }

// response
{
  "currency": "GBP",
  "price_total": 387.50,
  "deposit_amount": null,
  "amount_due_now": 387.50,    // deposit if set, else full; 0 if no payment needed
  "require_payment": true,
  "breakdown": [ { "label": "Hot food × 10", "amount": 387.50 } ]
}
```

Render `breakdown` as the line items and `amount_due_now` as the amount you are about to charge.

POST /bookings/v2/guest

PUBLIC

Creates the booking. **Rate limited to 10 requests per minute per IP.**

```
{
  "store_id": 42,
  "service_id": 1,
  "start_at": "2026-09-15T15:00:00",    // exactly as returned by /availability
  "party_size": 8,
  "customer_name": "Sarah Ahmed",
  "customer_phone": "07700 900123",
  "customer_email": "sarah@example.com",
  "note": "Arriving 10 minutes early",
  "option_ids": [],
  "form_data": {                      // FREE-FORM – see below
    "child_name": "Yusuf",
    "child_dob": "2018-04-02",
    "child_favourite_team": "Arsenal",
    "is_birthday": true
  }
}
```

`form_data` is deliberately schema-free

Put whatever your booking form collects in here — the backend stores it as-is. Adding a question needs no backend change. Three keys are special: `child_name`, `child_dob` and `child_favourite_team` are picked up automatically and printed in the confirmation email. Keep those exact names for the football venue.

The response is the created booking. **Two fields matter most:**

FIELD	WHY IT MATTERS
<code>id</code>	needed for the payment call
<code>reference</code>	e.g. <code>BK-7KQ2WM</code> — the customer's proof of ownership. Required to pay or to amend later. Show it on the confirmation screen and keep it in the URL of any "manage my booking" link.

6 · Taking payment with Stripe

POST `/bookings/v2/{booking_id}/payment-intent?reference=BK-XXXXXX`

PUBLIC

```
{
  "booking_id": 1,
  "client_secret": "pi_3abc..._secret_xyz",
  "amount": 150.00,
  "currency": "GBP",
  "payment_status": "pending"
}
```

Standard Stripe Payment Element flow — nothing custom:

```
// 1. create the booking, keep id + reference
const booking = await api.post('/bookings/v2/guest', payload);

// 2. ask for a payment intent
const { client_secret } = await api.post(
  `/bookings/v2/${booking.id}/payment-intent?reference=${booking.reference}`
);

// 3. mount Stripe Elements with client_secret, then confirm
const { error } = await stripe.confirmPayment({
  elements,
  confirmParams: { return_url: `https://tekkersuk.com/booking/${booking.reference}` }
});

// 4. on the return page, poll until the webhook has confirmed it
const b = await api.get(
  `/bookings/v2/guest/${id}?reference=${reference}`
);
if (b.payment_status === 'paid') showConfirmed(b);
```

Do not treat the browser redirect as confirmation.

Stripe's `return_url` fires before our webhook may have landed. On the return page, read the booking and show "confirmed" only once `payment_status` is `"paid"`. If it is still `"pending"` after a few seconds, show "Payment received, confirming your booking..." rather than an error — the webhook is authoritative and will catch up.

PAYMENT_STATUS	MEANING
not_required	free booking, nothing to pay
pending	awaiting payment or webhook
paid	confirmed — booking becomes booked
failed	card declined; the customer may retry on the same booking

Currency is per venue.

Tekkers is GBP. Never hardcode a symbol — read currency from /quote or the booking.

7 · Handling rejections

A refused booking returns **409** with a structured body. This is normal, not an error state — someone else may have taken the slot while your customer filled the form.

```
{ "detail": {
  "reason": "slot_full",
  "available": 0,
  "alternatives": [
    { "time": "16:30", "datetime": "2026-09-15T16:30:00", "bookable": true }
  ]
} }
```

Always render alternatives as tappable options. It turns a dead end into a one-tap recovery.

REASON	CAUSE & WHAT TO DO
slot_full	taken while they were deciding — offer alternatives
invalid_slot	the time isn't on the published grid; re-fetch availability
too_soon	inside the lead time (1h for Tekkers) — "please call us"
party_too_large / party_too_small	headcount outside the package limits
closed / closed_holiday	venue shut that day

8 · Staff & owner endpoints

All require a store-owner or admin JWT in Authorization: Bearer ...

GET /bookings/v2/store/today?store_id={id}

OWNER

The dashboard call. Returns today's bookings plus a summary block: total, pending, booked, cancelled, guests, revenue_booked, revenue_paid.

PUT /bookings/v2/{booking_id}

OWNER

Accept, decline, move or edit. {"status":"booked"} confirms; {"status":"cancelled"} declines. Add customer_message to include a line in the email the customer receives. Moving the time re-checks capacity and may 409.

POST /bookings/v2/store/filter

OWNER

Body: { "store_id": 42, "target_date": "2026-09-15", "limit": 100, "offset": 0 }. Omit target_date for all dates.

MANAGING PACKAGES

ENDPOINT	PURPOSE
GET PUT /config?store_id=	venue settings: currency, lead time, booking window, on/off
GET POST /resources PUT DELETE /resources/{id}	the stadium / chairs / rooms
POST /services PUT DELETE /services/{id}	the packages, their options and hours

Deleting is intentionally restricted.

A service with bookings cannot be deleted — set `is_active: false` instead. Same for a resource still used by a service. This protects booking history.

CUSTOMER SELF-SERVICE (NO LOGIN)

GET /bookings/v2/guest/{booking_id}?reference=BK-XXXXXX

PUBLIC

Read a booking with no login — this is what the confirmation screen polls after Stripe redirects back. Rate limited to 30/min. A wrong reference returns **404**, not 403, so codes can't be probed.

PUT /bookings/v2/guest/{booking_id}?reference=BK-XXXXXX

PUBLIC

Lets a guest amend or cancel from an email link. They may change time, headcount, name, phone, note, and `form_data` — and may only set status to `"cancelled"`. Duration and price stay with the venue.

9 • Gotchas

1. **Send back the server's exact `datetime` string.** Rebuilding it from `time` plus your own date, or letting a JS `Date` add a timezone offset, produces an off-grid value and a 409 `invalid_slot`. Treat the string as opaque.
2. **No timezone suffix.** Times are Berlin wall-clock. Never send `...T15:00:00Z` or `+01:00`.
3. **Never show `buffer_minutes` to the customer.** It exists to block the next slot, not to describe their session.
4. **The reference code is the guest's only credential.** No reference means no paying and no amending. Put it in the confirmation email, the success screen, and any manage-booking URL.
5. **Re-fetch availability after a 409.** The grid you are holding is stale.
6. **Don't compute totals client-side for display-then-charge.** Use `/quote`. Per-person pricing with options is easy to get subtly wrong.
7. **All three Tekkers packages share one stadium.** If you build a "see all packages for this date" view, call `/availability` per service — they will legitimately block each other.
8. **Bookings start `pending`, always.** Even after successful payment the confirmation comes from the webhook. Poll rather than assume.

Suggested build order

1. Package list from `/services` — static, easy, proves connectivity.
2. Date picker + slot grid from `/availability`.
3. Booking form → `POST /guest`, handle 409 with alternatives.

4. Stripe Payment Element + the confirmation-polling screen.

5. Manage-booking page using the reference code.

Test the concurrency case early.

Open two browsers, hold the same slot, submit both. The second must get a 409 with alternatives, not a double booking. The backend serialises this with a row lock — make sure your UI recovers gracefully.

Backend contact: mandeep783@gmail.com · Endpoints implemented in `app/routes/booking_v2_routes.py`; availability rules in `app/services/booking_availability_service.py`. Live API docs (FastAPI, needs the server running): `/docs` → tag **Bookings v2**.